SENATI

Escuela de Tecnologías de la Información

GUÍA DE LABORATORIO

Guía 09

Aprendizaje supervisado: Regresión Lineal

Crea programas con algoritmos de aprendizaje supervisado

Escuela	Tecnologías de la Información
Carrera	Ingeniería de Software con Inteligencia Artificial
Curso	Seminario de Complementación Práctica I (PIAD-427)
Semana	9
Tarea de aprendizaje	Crea programas con algoritmos de aprendizaje supervisado
Herramientas	Python 3 · Visual Studio Code · scikit-learn
Instructor	Rembrandt Ubalde Enríquez
Estudiante	
Fecha	

Contenido

Objetivos de la guía 3
Materiales 3
Parte 1. Preparación del entorno en Visual Studio Code 4
1.1 Crear la carpeta del proyecto 4
1.2 Crear y activar el entorno virtual 4
1.3 Instalar las librerías 4
1.4 Seleccionar el intérprete y ejecutar scripts 4
Parte 2. Conceptos clave 6
2.1 ¿Qué es el aprendizaje supervisado? 6
2.2 Tipos de algoritmos supervisados 6
2.3 Regresión lineal simple 6
2.4 Regresión lineal múltiple y tipos de regresión lineal 7
2.5 Etapas para implementar el modelo 7
Parte 3. Prácticas en Visual Studio Code 9
Práctica 01. Clasificación vs. regresión 9
Práctica 02. Preparación de datos 9
Práctica 03. Regresión lineal simple «a mano» 11
Práctica 04. Regresión lineal simple con scikit-learn 12
Práctica 05. Regresión lineal múltiple (conocimiento complementario) 14
Práctica 06. Guardar y reutilizar el modelo 16
Parte 4. Errores frecuentes 16
Parte 5. Ejercicios propuestos 17
Preguntas de reflexión 17
Lista de cotejo 17

(Si el índice aparece vacío, haz clic derecho sobre él y elige «Actualizar campos».)

Objetivos de la guía

Al terminar esta guía serás capaz de:

- **Describir** los tipos de algoritmos del aprendizaje supervisado: clasificación y regresión.
- **Definir** la regresión lineal simple y la regresión lineal múltiple.
- **Implementar** el algoritmo de regresión lineal simple con Python en Visual Studio Code, siguiendo sus cuatro etapas: preparación de datos, entrenamiento, predicción y evaluación.

Operación	Dónde se trabaja en esta guía
Describe los tipos de algoritmos del aprendizaje supervisado	Parte 2 (sección 2.1 y 2.2) y Práctica 01
Define la regresión lineal simple y múltiple	Parte 2 (secciones 2.3 a 2.5), Prácticas 03 y 05
Implementa algoritmo de regresión lineal simple con Python	Parte 3 (Prácticas 02, 04 y 06)

Materiales

- Computadora con Windows 10/11, Python 3.10 o superior y Visual Studio Code con las extensiones **Python** y **Pylance** de Microsoft.
- Conexión a internet para instalar las librerías (solo la primera vez).

Parte 1. Preparación del entorno en Visual Studio Code

Si ya trabajaste las guías anteriores, tienes Python y VS Code listos. Igual sigue estos pasos para crear la **carpeta de esta guía** y su **entorno virtual**.

1.1 Crear la carpeta del proyecto

1. Crea en Documentos una carpeta llamada **IA_Guia09_Regresion_Lineal**.
2. Abre Visual Studio Code y entra a **Archivo > Abrir carpeta…** (Ctrl + K, Ctrl + O). Selecciona la carpeta.
3. Si VS Code pregunta **«¿Confía en los autores de los archivos?»**, elige **Sí, confío en los autores**.
4. Abre la terminal integrada con **Terminal > Nuevo terminal** (Ctrl + ñ).

1.2 Crear y activar el entorno virtual

Terminal de VS Code

```
python -m venv venv
.\venv\Scripts\activate
```

Cuando el entorno está activo, la terminal muestra **(venv)** al inicio de la línea.

> ⚠ **Atención:** Si PowerShell muestra **«la ejecución de scripts está deshabilitada en este sistema»**, ejecuta una sola vez `Set-ExecutionPolicy -Scope CurrentUser RemoteSigned`, responde **S** y vuelve a activar el entorno.

1.3 Instalar las librerías

Terminal de VS Code

```
pip install numpy pandas matplotlib scikit-learn joblib
```

Verifica la instalación:

Terminal de VS Code

```
python -c "import sklearn, pandas; print(sklearn.__version__, pandas.__version__)"
```

▶ Resultado esperado en la terminal

```
1.x.x 2.x.x   (o versiones más recientes)
```

1.4 Seleccionar el intérprete y ejecutar scripts

1. Pulsa **Ctrl + Shift + P**, escribe **Python: Select Interpreter** y elige el que dice **('venv': venv)**.
2. Crea cada archivo con el ícono **Nuevo archivo** del Explorador, con el nombre exacto que indica la guía (por ejemplo `01_tipos_supervisado.py`).
3. Copia el código y guarda con **Ctrl + S**.

4. Ejecuta con el botón ▶ **Run Python File** (arriba a la derecha) o escribiendo `python nombre_del_archivo.py` en la terminal.
5. Compara lo que aparece en la terminal con el recuadro verde **Resultado esperado**.

> 💡 **Consejo:** Cuando un script usa `plt.show()`, el gráfico se abre en una **ventana aparte** y el programa queda en pausa hasta que **cierres esa ventana**. El gráfico también queda guardado como imagen PNG en tu carpeta gracias a `plt.savefig(...)`.

> 💡 **Consejo:** Para depurar paso a paso: haz clic a la izquierda del número de línea para poner un **punto de interrupción** (punto rojo) y pulsa **F5 > Python File**. En el panel **Variables** verás los valores de `X`, `y`, `modelo`, etc. mientras el programa avanza.

Parte 2. Conceptos clave

2.1 ¿Qué es el aprendizaje supervisado?

En el **aprendizaje supervisado** el modelo aprende a partir de ejemplos que ya tienen la **respuesta correcta** (la **etiqueta**). Es como un estudiante que practica con un solucionario: ve muchas preguntas con su respuesta, descubre el patrón y luego responde preguntas nuevas por su cuenta.

Término	Significado	Ejemplo
Variables de entrada (X)	Datos que conocemos y usamos para predecir. También se llaman **características** o features.	Horas de estudio
Variable objetivo (y)	Lo que queremos predecir. Es la **etiqueta**.	Nota del examen
Entrenamiento	El modelo ajusta sus parámetros para que sus predicciones se parezcan a y.	`modelo.fit(X, y)`
Predicción	El modelo entrenado responde con datos nuevos.	`modelo.predict(X_nuevo)`

2.2 Tipos de algoritmos supervisados

	Clasificación	Regresión
¿Qué predice?	Una **categoría** (clase) de un conjunto finito.	Un **número continuo**.
Pregunta típica	¿A qué grupo pertenece?	¿Cuánto? ¿Qué valor tendrá?
Ejemplos	Correo spam / no spam; especie de una flor; aprobado / desaprobado.	Precio de un departamento; nota de un examen; ventas del próximo mes.
Algoritmos	KNN, Árbol de decisión, Regresión logística, SVM, Random Forest.	Regresión lineal simple y múltiple, Árbol de regresión, SVR.
Métricas	Exactitud (accuracy), precisión, recall, matriz de confusión.	MAE, MSE, RMSE, R^2.

> ⚠ **Atención:** La **regresión logística**, a pesar de su nombre, es un algoritmo de **clasificación**: predice una probabilidad y con ella asigna una clase.

2.3 Regresión lineal simple

Busca la **recta** que mejor representa la relación entre **una** variable de entrada x y una variable objetivo y:

Ecuación del modelo

```
ŷ = b0 + b1 · x

b0 = intercepto  (valor de ŷ cuando x = 0)
b1 = pendiente   (cuánto cambia ŷ cuando x aumenta en 1)
```

Para hallar b0 y b1 se usa el método de **mínimos cuadrados**: se elige la recta que hace **lo más pequeña posible** la suma de los errores al cuadrado entre el valor real y y el valor predicho ŷ.

Fórmulas de mínimos cuadrados

```
b1 = Σ (x - x̄)(y - ȳ)  /  Σ (x - x̄)²
b0 = ȳ - b1 · x̄

x̄, ȳ = promedios de x y de y
```

2.4 Regresión lineal múltiple y tipos de regresión lineal

Cuando la predicción depende de **varias** variables de entrada, se usa la **regresión lineal múltiple**: `ŷ = b0 + b1·x1 + b2·x2 + … + bn·xn`. Cada coeficiente indica cuánto cambia ŷ cuando su variable aumenta en 1 y las demás se mantienen iguales.

Tipo	Idea	Aplicación
Simple	Una variable de entrada, una recta.	Nota según horas de estudio.
Múltiple	Varias variables de entrada, un plano o hiperplano.	Precio de un departamento según área, habitaciones y antigüedad.
Polinómica	Agrega potencias (x^2, x^3) para curvas, pero sigue siendo lineal en sus coeficientes.	Crecimiento de ventas que se acelera.
Regularizada (Ridge, Lasso)	Penaliza coeficientes muy grandes para evitar el sobreajuste.	Muchas variables con poca cantidad de datos.

Aplicaciones de la regresión lineal múltiple: estimar precios de inmuebles, proyectar ventas según publicidad y temporada, predecir consumo de energía según temperatura y horario, estimar el tiempo de entrega según distancia y tráfico.

2.5 Etapas para implementar el modelo

Etapa	Qué se hace	Código clave
1. Preparación de datos	Cargar, revisar nulos, explorar, graficar y dividir en entrenamiento y prueba.	`train_test_split`
2. Entrenamiento del modelo	El algoritmo calcula la pendiente y el intercepto.	`modelo.fit()`
3. Realización de predicciones	El modelo estima valores para datos que no vio.	`modelo.predict()`
4. Evaluación del modelo	Se compara lo predicho con lo real mediante métricas.	`r2_score`, `mean_absolute_error`

Métrica	Interpretación
MAE	Error promedio en las mismas unidades de y. Si MAE = 0.9, el modelo se equivoca en promedio 0.9 puntos de nota.
RMSE	Parecido al MAE, pero castiga más los errores grandes.

Métrica	Interpretación
R^2	Qué tanto de la variación de y explica el modelo. Va de 0 a 1 (más cerca de 1 es mejor).

Parte 3. Prácticas en Visual Studio Code

Práctica 01. Clasificación vs. regresión

Comprobarás la diferencia entre los dos tipos de algoritmos supervisados: uno devuelve una **categoría** y el otro un **número**.

Archivo: 01_tipos_supervisado.py

```
# 01_tipos_supervisado.py - Clasificación vs. Regresión (aprendizaje supervisado)
from sklearn.datasets import load_iris
from sklearn.neighbors import KNeighborsClassifier
from sklearn.linear_model import LinearRegression

# ---- 1. CLASIFICACIÓN: la salida es una CATEGORÍA ----
iris = load_iris()
X, y = iris.data, iris.target                  # y = especie (0, 1 o 2)
clasificador = KNeighborsClassifier(n_neighbors=3)
clasificador.fit(X, y)                          # aprende con datos ETIQUETADOS
flor_nueva = [[5.1, 3.5, 1.4, 0.2]]
especie = clasificador.predict(flor_nueva)[0]
print("CLASIFICACIÓN")
print("  Medidas de la flor:", flor_nueva[0])
print("  Especie predicha  :", iris.target_names[especie])

# ---- 2. REGRESIÓN: la salida es un NÚMERO continuo ----
horas = [[1], [2], [3], [4], [5]]               # entrada: horas de estudio
notas = [8, 10, 12, 14, 16]                     # salida: nota obtenida
regresor = LinearRegression()
regresor.fit(horas, notas)
print("\nREGRESIÓN")
print("  Horas de estudio  : 6")
print("  Nota predicha     :", round(regresor.predict([[6]])[0], 2))
```

▶ Resultado esperado en la terminal

```
CLASIFICACIÓN
  Medidas de la flor: [5.1, 3.5, 1.4, 0.2]
  Especie predicha  : setosa

REGRESIÓN
  Horas de estudio  : 6
  Nota predicha     : 18.0
```

Práctica 02. Preparación de datos

Generarás un dataset de 40 estudiantes (horas de estudio y nota), lo guardarás como **estudiantes.csv** y lo explorarás. Este archivo se usa en las prácticas siguientes, así que **ejecuta esta práctica primero**.

Archivo: 02_preparar_datos.py

```
# 02_preparar_datos.py - Preparación de datos para la regresión lineal
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

np.random.seed(42)                                  # misma "aleatoriedad" en todas las PC
horas = np.round(np.random.uniform(1, 10, 40), 1)
ruido = np.random.normal(0, 1.2, 40)
nota = np.clip(np.round(5 + 1.4 * horas + ruido, 1), 0, 20)

df = pd.DataFrame({"horas_estudio": horas, "nota": nota})
df.to_csv("estudiantes.csv", index=False)        # guardamos el dataset

print("Primeras filas:")
print(df.head())
print("\nTamaño del dataset:", df.shape)
print("Valores nulos por columna:")
print(df.isnull().sum())
print("\nEstadísticas descriptivas:")
print(df.describe().round(2))
print("\nCorrelación horas-nota:", round(df["horas_estudio"].corr(df["nota"]), 3))

plt.scatter(df["horas_estudio"], df["nota"], color="steelblue")
plt.title("Horas de estudio vs. Nota")
plt.xlabel("Horas de estudio")
plt.ylabel("Nota (0 a 20)")
plt.grid(alpha=0.3)
plt.savefig("g9_dispersion.png", dpi=110, bbox_inches="tight")
plt.show()
```

▶ Resultado esperado en la terminal

```
Primeras filas:
   horas_estudio  nota
0            4.4  11.1
1            9.6  17.2
2            7.6  16.6
3            6.4  12.5
4            2.4   8.6

Tamaño del dataset: (40, 2)
Valores nulos por columna:
horas_estudio    0
nota             0
dtype: int64

Estadísticas descriptivas:
       horas_estudio   nota
count          40.00  40.00
mean            5.11  12.05
std             2.67   3.93
min             1.20   5.50
25%             2.68   8.90
50%             4.95  11.70
75%             7.25  15.20
max             9.70  19.90

Correlación horas-nota: 0.963
```

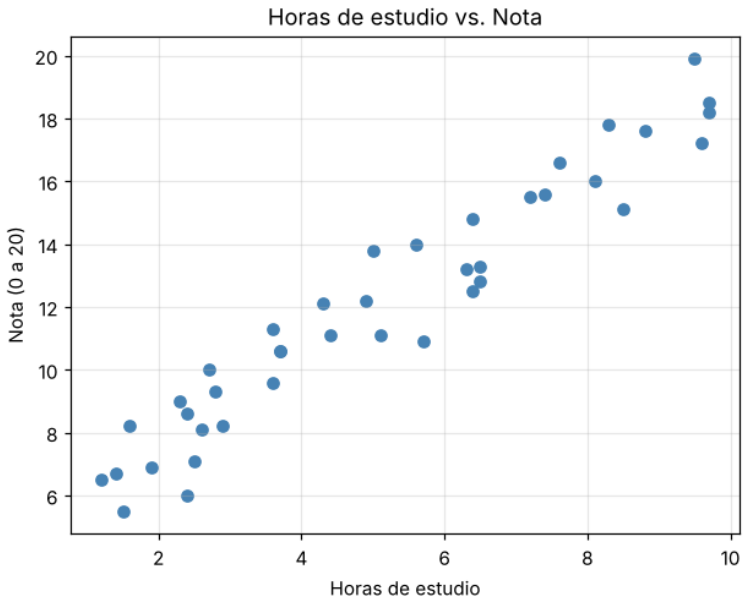


Figura 1. Diagrama de dispersión: la nube de puntos sube de forma casi recta.

> 💡 **Consejo:** Una correlación de **0.963** (cercana a 1) indica una relación lineal positiva muy fuerte: tiene sentido usar regresión lineal.

Práctica 03. Regresión lineal simple «a mano»

Antes de usar scikit-learn, calcula la pendiente y el intercepto con las fórmulas de mínimos cuadrados de la sección 2.3.

Archivo: 03_regresion_manual.py

```
# 03_regresion_manual.py - Regresión lineal simple calculada "a mano" (mínimos
cuadrados)
import pandas as pd

df = pd.read_csv("estudiantes.csv")
x = df["horas_estudio"]
y = df["nota"]

x_media, y_media = x.mean(), y.mean()
# Pendiente:   b1 = Σ(x - x̄)(y - ȳ) / Σ(x - x̄)²
b1 = ((x - x_media) * (y - y_media)).sum() / ((x - x_media) ** 2).sum()
# Intercepto:  b0 = ȳ - b1·x̄
b0 = y_media - b1 * x_media

print(f"Pendiente  (b1): {b1:.4f}")
print(f"Intercepto (b0): {b0:.4f}")
print(f"Ecuación: nota = {b0:.2f} + {b1:.2f} * horas")
for h in [2, 5, 8]:
    print(f"  Si estudia {h} h -> nota estimada = {b0 + b1 * h:.2f}")
```

▶ **Resultado esperado en la terminal**

```
Pendiente  (b1): 1.4194
Intercepto (b0): 4.8006
Ecuación: nota = 4.80 + 1.42 * horas
  Si estudia 2 h -> nota estimada = 7.64
  Si estudia 5 h -> nota estimada = 11.90
  Si estudia 8 h -> nota estimada = 16.16
```

Interpretación: por cada hora adicional de estudio, la nota sube en promedio **1.42 puntos**; un estudiante que no estudia obtendría alrededor de **4.80**.

Práctica 04. Regresión lineal simple con scikit-learn

Este es el flujo completo del algoritmo con sus cuatro etapas. Observa los comentarios numerados en el código.

Archivo: 04_regresion_sklearn.py

```
# 04_regresion_sklearn.py - Regresión lineal simple con scikit-learn (flujo completo)
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score

# 1. PREPARACIÓN DE DATOS
df = pd.read_csv("estudiantes.csv")
X = df[["horas_estudio"]]          # doble corchete: X debe ser 2D (tabla)
y = df["nota"]                     # y es 1D (columna)
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42)
print("Datos de entrenamiento:", len(X_train), "| Datos de prueba:", len(X_test))

# 2. ENTRENAMIENTO DEL MODELO
modelo = LinearRegression()
modelo.fit(X_train, y_train)
print(f"Pendiente  : {modelo.coef_[0]:.4f}")
print(f"Intercepto : {modelo.intercept_:.4f}")

# 3. REALIZACIÓN DE PREDICCIONES
y_pred = modelo.predict(X_test)
comparacion = pd.DataFrame({"horas": X_test["horas_estudio"].values,
                            "nota_real": y_test.values,
                            "nota_predicha": y_pred.round(2)})
print("\nComparación en datos de prueba:")
print(comparacion.to_string(index=False))

# 4. EVALUACIÓN DEL MODELO
mae = mean_absolute_error(y_test, y_pred)
rmse = np.sqrt(mean_squared_error(y_test, y_pred))
r2 = r2_score(y_test, y_pred)
print(f"\nMAE  (error absoluto medio): {mae:.3f}")
print(f"RMSE (raíz del error cuadr.): {rmse:.3f}")
print(f"R²   (coef. de determinación): {r2:.3f}")

# Gráfico: puntos reales y recta aprendida
plt.scatter(X_train, y_train, color="steelblue", label="Entrenamiento")
plt.scatter(X_test, y_test, color="orange", label="Prueba")
xs = pd.DataFrame({"horas_estudio": np.linspace(1, 10, 50)})
plt.plot(xs, modelo.predict(xs), color="red", linewidth=2, label="Recta de regresión")
plt.title("Regresión lineal simple: horas vs. nota")
plt.xlabel("Horas de estudio"); plt.ylabel("Nota")
plt.legend(); plt.grid(alpha=0.3)
plt.savefig("g9_recta.png", dpi=110, bbox_inches="tight")
plt.show()
```

▶ **Resultado esperado en la terminal**

```
Datos de entrenamiento: 32 | Datos de prueba: 8
Pendiente  : 1.4619
Intercepto : 4.5265

Comparación en datos de prueba:
 horas  nota_real  nota_predicha
   3.6        9.6           9.79
   3.7       10.6           9.94
   2.7       10.0           8.47
   2.8        9.3           8.62
   2.4        8.6           8.04
   8.5       15.1          16.95
   1.9        6.9           7.30
   5.6       14.0          12.71

MAE  (error absoluto medio): 0.896
RMSE (raíz del error cuadr.): 1.051
R²   (coef. de determinación): 0.832
```

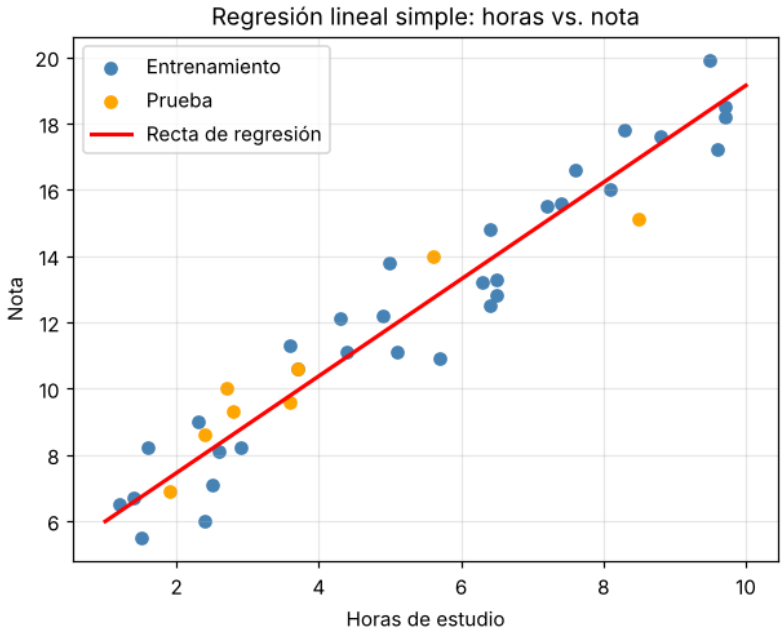


Figura 2. Recta de regresión aprendida (rojo) sobre los datos de entrenamiento y de prueba.

La pendiente (1.46) es un poco distinta a la de la Práctica 03 (1.42) porque aquí el modelo se entrena **solo con el 80 %** de los datos. Con un **R^2 de 0.832** el modelo explica el 83 % de la variación de las notas en datos que nunca vio.

> ⚠ **Atención:** `X` debe ser una tabla (2D): por eso se escribe `df[["horas_estudio"]]` con **doble corchete**. Con un solo corchete aparece el error **«Expected 2D array, got 1D array instead»**.

Práctica 05. Regresión lineal múltiple (conocimiento complementario)

Predecirás el precio de un departamento usando **tres** variables a la vez.

Archivo: 05_regresion_multiple.py

```
# 05_regresion_multiple.py - Regresión lineal MÚLTIPLE: precio de departamentos
import numpy as np
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import r2_score, mean_absolute_error

np.random.seed(7)
n = 120
area = np.random.randint(45, 160, n)             # m²
habitaciones = np.random.randint(1, 5, n)
antiguedad = np.random.randint(0, 30, n)         # años
precio = (2500 * area + 15000 * habitaciones - 1800 * antiguedad
          + 40000 + np.random.normal(0, 20000, n))  # soles
df = pd.DataFrame({"area": area, "habitaciones": habitaciones,
                   "antiguedad": antiguedad, "precio": precio.round(0)})
print(df.head())

X = df[["area", "habitaciones", "antiguedad"]]   # VARIAS variables de entrada
y = df["precio"]
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.25,
random_state=1)

modelo = LinearRegression().fit(X_train, y_train)
print("\nCoeficientes (cuánto cambia el precio por cada unidad):")
for nombre, coef in zip(X.columns, modelo.coef_):
    print(f"  {nombre:<13}: {coef:>10.2f}")
print(f"  Intercepto   : {modelo.intercept_:>10.2f}")

y_pred = modelo.predict(X_test)
print(f"\nR²  : {r2_score(y_test, y_pred):.3f}")
print(f"MAE : S/ {mean_absolute_error(y_test, y_pred):,.2f}")

nuevo = pd.DataFrame({"area": [90], "habitaciones": [3], "antiguedad": [10]})
print(f"\nDepartamento de 90 m², 3 hab., 10 años -> S/
{modelo.predict(nuevo)[0]:,.2f}")
```

▶ Resultado esperado en la terminal

```
   area  habitaciones  antiguedad    precio
0    92             4           9  303297.0
1   113             4           9  379380.0
2    70             4          13  254698.0
3   112             2          21  286159.0
4   128             3          11  404020.0

Coeficientes (cuánto cambia el precio por cada unidad):
  area         :    2592.34
  habitaciones :   17724.84
  antiguedad   :   -1664.93
  Intercepto   :   25015.32

R²  : 0.900
MAE : S/ 17,486.96

Departamento de 90 m², 3 hab., 10 años -> S/ 294,851.06
```

Interpretación: cada m² adicional suma unos **S/ 2 592** al precio; cada habitación, unos **S/ 17 725**; cada año de antigüedad lo reduce en unos **S/ 1 665**. El signo negativo indica una relación inversa.

Práctica 06. Guardar y reutilizar el modelo

En un sistema real el modelo se entrena una vez y luego se reutiliza. Con **joblib** se guarda en un archivo `.pkl`.

Archivo: 06_guardar_modelo.py

```
# 06_guardar_modelo.py - Guardar el modelo entrenado y reutilizarlo
import joblib
import pandas as pd
from sklearn.linear_model import LinearRegression

df = pd.read_csv("estudiantes.csv")
modelo = LinearRegression().fit(df[["horas_estudio"]], df["nota"])
joblib.dump(modelo, "modelo_notas.pkl")            # se guarda en un archivo
print("Modelo guardado en modelo_notas.pkl")

modelo_cargado = joblib.load("modelo_notas.pkl")   # se vuelve a cargar
horas = 7.5                                        # en clase: float(input("Horas: "))
nota = modelo_cargado.predict(pd.DataFrame({"horas_estudio": [horas]}))[0]
print(f"Con {horas} horas de estudio, la nota estimada es {nota:.2f}")
print("¿Aprueba (nota >= 10.5)?", "Sí" if nota >= 10.5 else "No")
```

▶ Resultado esperado en la terminal

```
Modelo guardado en modelo_notas.pkl
Con 7.5 horas de estudio, la nota estimada es 15.45
¿Aprueba (nota >= 10.5)? Sí
```

> 💡 **Consejo:** Reemplaza `horas = 7.5` por `horas = float(input("Horas: "))` para que el usuario escriba el valor en la terminal.

Parte 4. Errores frecuentes

Mensaje o síntoma	Causa	Solución
`ModuleNotFoundError: No module named 'sklearn'`	La librería no está instalada en el entorno activo.	Activa el venv y ejecuta `pip install scikit-learn`. Revisa el intérprete seleccionado.
`FileNotFoundError: estudiantes.csv`	No se ejecutó la Práctica 02 o la terminal está en otra carpeta.	Ejecuta `02_preparar_datos.py` y confirma con `dir` que el CSV está en la carpeta.
`Expected 2D array, got 1D array instead`	Se pasó X con un solo corchete.	Usa `df[["columna"]]`.
«No se encontró Python; ejecutar sin argumentos para instalar desde Microsoft Store»	Python no está instalado o no está en el PATH.	Instala Python desde python.org marcando **Add python.exe to PATH** y reinicia VS Code.

Mensaje o síntoma	Causa	Solución
La terminal se queda «colgada» después de un gráfico	La ventana de `plt.show()` sigue abierta.	Cierra la ventana del gráfico.
R^2 negativo o muy bajo	Los datos no tienen relación lineal o hay errores en los datos.	Grafica la dispersión y revisa la correlación antes de entrenar.

Parte 5. Ejercicios propuestos

1. Cambia `test_size` a 0.3 en la Práctica 04. ¿Cómo cambian la pendiente y el R^2? Explica por qué.
2. Crea un dataset de 30 registros con **temperatura (°C)** y **ventas de helados**. Entrena una regresión lineal simple y predice las ventas para 28 °C.
3. En la Práctica 05 agrega la variable `distrito_premium` (0 o 1) al cálculo del precio. ¿Mejora el R^2?
4. Modifica la Práctica 06 para pedir las horas con `input()` y mostrar un mensaje si el valor no está entre 0 y 12.
5. Compara el MAE de un modelo que siempre predice el promedio de las notas con el MAE de tu regresión. ¿Cuánto mejora?

Preguntas de reflexión

1. ¿Por qué la regresión lineal es un algoritmo **supervisado**?
2. ¿Qué significa en la vida real una pendiente negativa?
3. ¿Por qué se evalúa el modelo con datos de **prueba** y no con los de entrenamiento?
4. Menciona dos problemas de tu entorno laboral que se puedan resolver con regresión lineal múltiple.

Lista de cotejo

- ☐ Creé la carpeta del proyecto, el entorno virtual y seleccioné el intérprete **venv**.
- ☐ Diferencio clasificación y regresión con ejemplos propios.
- ☐ Generé y exploré **estudiantes.csv** (Práctica 02).
- ☐ Calculé la pendiente y el intercepto con las fórmulas (Práctica 03).
- ☐ Entrené, predije y evalué con scikit-learn, e interpreté MAE, RMSE y R^2 (Práctica 04).
- ☐ Implementé una regresión lineal múltiple e interpreté sus coeficientes (Práctica 05).
- ☐ Guardé y cargué el modelo con joblib (Práctica 06).
- ☐ Resolví al menos tres ejercicios propuestos.